

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 35%

QUESTIONS: 5 Duration : 1 Hour Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

*I **Nithya shree . k [1922521145]** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

NITHYA SHREE . K

Analytical Problem Solving**ANSWERS – Questions 5, 6, 9, 12 and 15****5. 8-Puzzle with Heuristic Search**

Start state

Goal state

2	8	3		1	2	3
1	6	4		8	_	4
7	_	5		7	6	5

i. Problem formulation

- **State:** a 3×3 arrangement of tiles 1–8 and the blank (a 9-element list).
- **Initial state:** 2 8 3 / 1 6 4 / 7 _ 5 (as given).
- **Actions:** move the blank Up, Down, Left or Right (only if it stays inside the board).
- **Transition model:** swap the blank with the adjacent tile in the chosen direction; result is the new board.
- **Goal test:** board equals 1 2 3 / 8 _ 4 / 7 6 5.
- **Path cost:** each move costs 1, so the cost of a solution = number of moves.

ii. Heuristics and values for the start state

h_1 = number of misplaced tiles (blank not counted). **h_2** = sum of Manhattan distances ($|\Delta\text{row}| + |\Delta\text{col}|$) of every tile from its goal position. Positions are written (row, column).

Tile	Start pos	Goal pos	Misplaced?	Manhattan dist.
1	(2,1)	(1,1)	Yes	1
2	(1,1)	(1,2)	Yes	1
3	(1,3)	(1,3)	No	0
4	(2,3)	(2,3)	No	0
5	(3,3)	(3,3)	No	0
6	(2,2)	(3,2)	Yes	1
7	(3,1)	(3,1)	No	0
8	(1,2)	(2,1)	Yes	2
Total			$h_1 = 4$	$h_2 = 5$

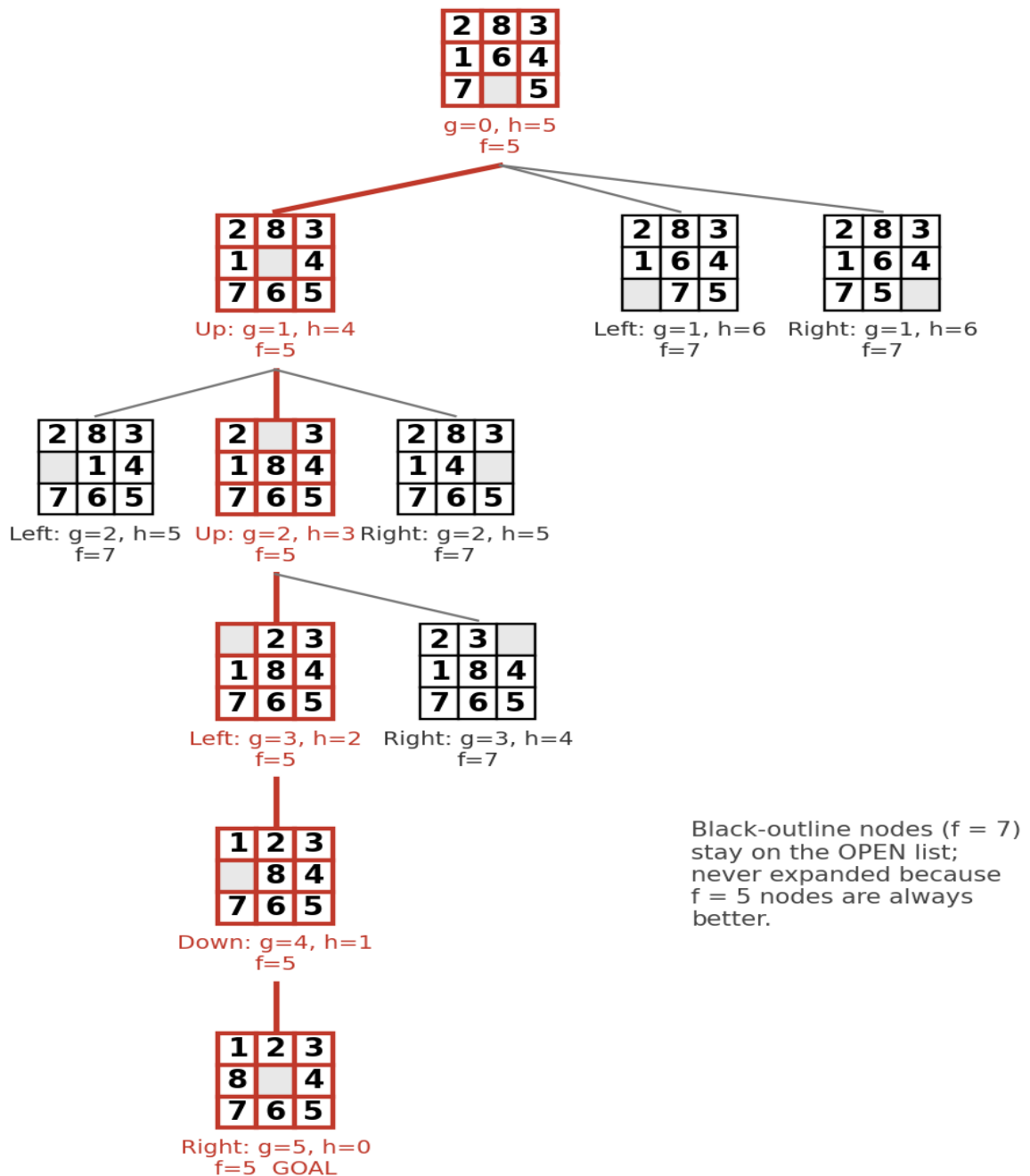
iii. A* search using h_2 ($f = g + h_2$)

Step	Node expanded	g	h_2	f	Children generated (move of blank : f)
1	Start	0	5	5	Up : 5 Left : 7 Right : 7
2	Up (A)	1	4	5	Up : 5 Left : 7 Right : 7
3	Up (B)	2	3	5	Left : 5 Right : 7
4	Left (C)	3	2	5	Down : 5
5	Down (D)	4	1	5	Right : 5 → GOAL ($h_2 = 0$)
6	Goal	5	0	5	Goal test succeeds – stop

Five $f = 7$ nodes (Left, Right at level 1; Left, Right at level 2; Right at level 3) stay on the OPEN list and are never expanded because the $f = 5$ nodes are always cheaper. A* expands only 5 nodes and generates 11.

Solution (optimal, 5 moves): Up → Up → Left → Down → Right (moves of the blank). Solution cost = 5. Search tree:

A* with h_2 (Manhattan): red path = solution, $f = g + h_2$



iv. Why h_2 dominates h_1

A misplaced tile is at least one move from its goal square, so its Manhattan distance is ≥ 1 . A correctly placed tile contributes 0 to both heuristics. Therefore $h_2(n) \geq h_1(n)$ for every state n , and both are admissible (neither overestimates the true number of moves). Since $h_2 \geq h_1$, every node that A* expands with h_2 is also expanded with h_1 , so h_2 expands no more nodes and is the better (more informed) heuristic, with the same optimal answer. Here $h_1 = 4$ but $h_2 = 5$, and h_2 is exactly equal to the true cost of 5.

6. Game Playing: Minimax with Alpha-Beta Pruning

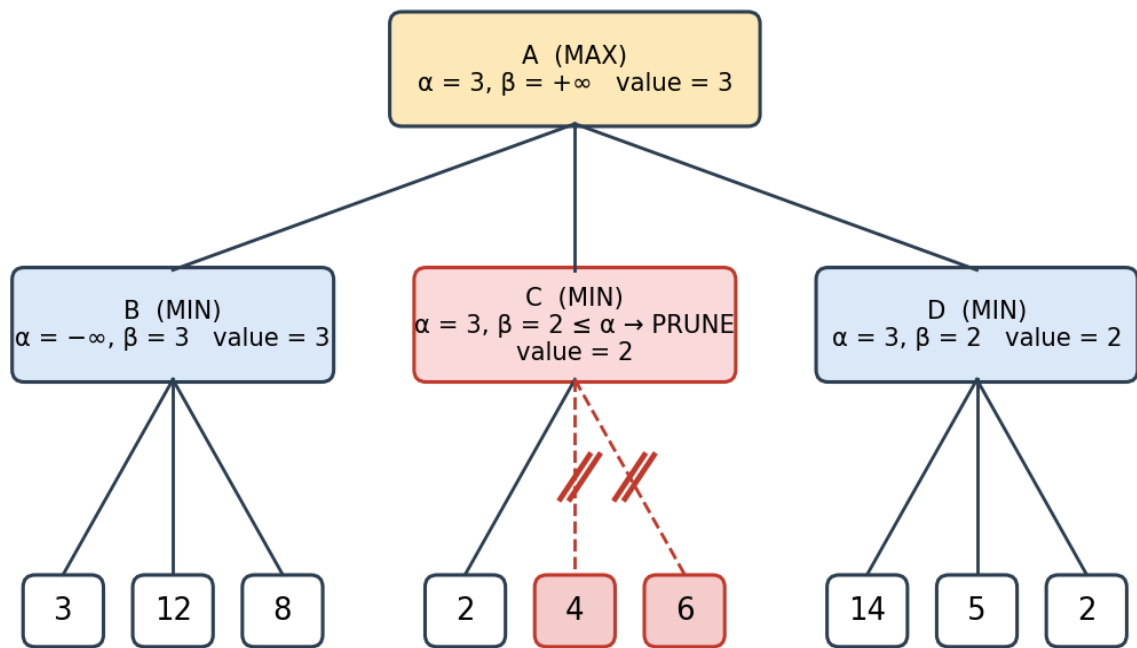
i. Minimax

MIN node	Leaf values	MIN value (smallest)
B	3, 12, 8	3
C	2, 4, 6	2
D	14, 5, 2	2

The MAX root picks the largest of the MIN values: $\max(3, 2, 2) = 3$. **Root value = 3 and the best move is to B.**

ii. Alpha-beta pruning

Node	α (in)	β (in)	What happens	Value
Root A (MAX)	$-\infty$	$+\infty$	Visit B first	—
B (MIN)	$-\infty$	$+\infty$	$3 \rightarrow \beta = 3$; $12 \rightarrow \beta$ stays 3; $8 \rightarrow \beta$ stays 3. No cut-off since $\beta > \alpha$	3
Root A	3	$+\infty$	α is updated to 3 (best for MAX so far)	—
C (MIN)	3	$+\infty$	First leaf $2 \rightarrow \beta = 2 \leq \alpha = 3 \rightarrow$ prune leaves 4 and 6	2
D (MIN)	3	$+\infty$	$14 \rightarrow \beta = 14$; $5 \rightarrow \beta = 5$; $2 \rightarrow \beta = 2 (\leq \alpha, \text{ but last leaf, nothing left to prune})$	2
Root A	3	$+\infty$	$\max(3, 2, 2) = 3$	3



Dashed red branches (leaves 4 and 6 under C) are pruned and never evaluated.

Result: root value = 3 (move to B). **Pruned branches:** the two leaves 4 and 6 under C. D is not pruned because its best-for-MIN leaf (2) comes last; 7 of 9 leaves are evaluated.

iii. Effect of move ordering

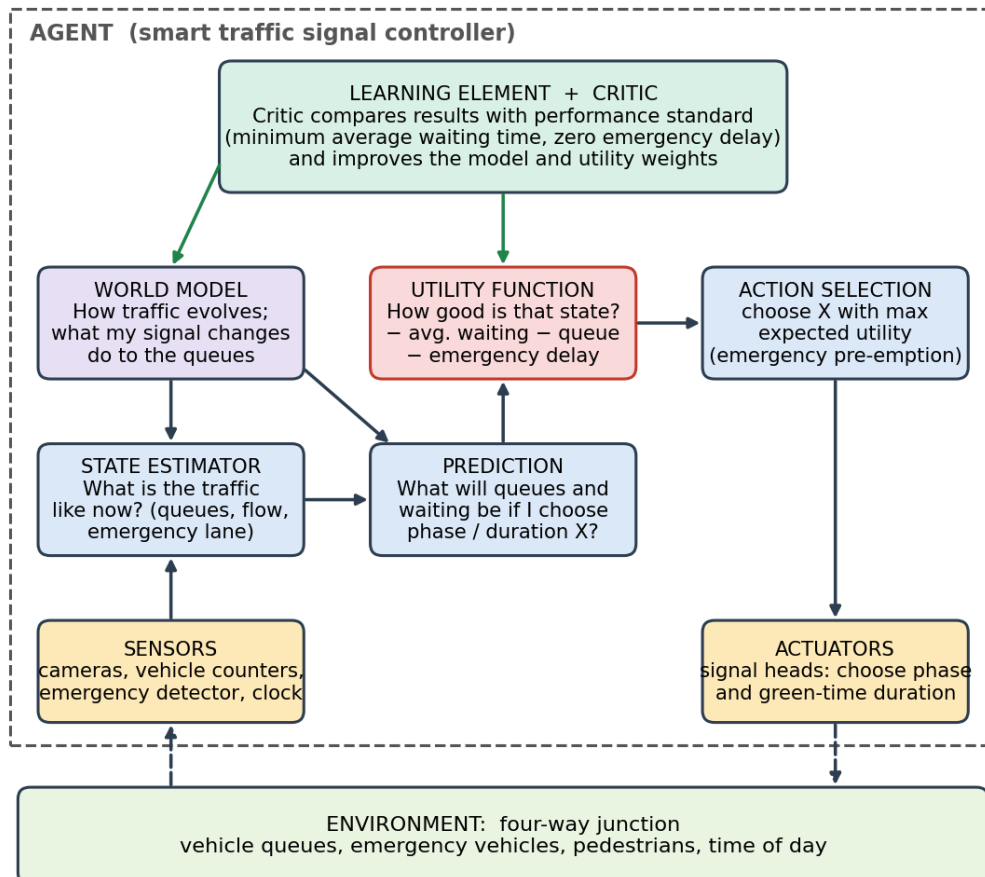
Alpha-beta always returns the same value as minimax, but the amount pruned depends on the order in which children are examined. If the best moves are examined first, cut-offs happen early: in D, examining 2 first would have pruned 14 and 5. With perfect ordering the number of nodes examined drops from $O(b^d)$ to about $O(b^{d/2})$, which lets the search go roughly twice as deep in the same time. With the worst ordering (best moves last) nothing is pruned and alpha-beta degenerates to plain minimax. In practice ordering is improved by iterative deepening, killer-move and history heuristics, and trying captures or previously best moves first.

9. Smart Traffic Signal Controller (Agent Design)

i. Most suitable agent type: Utility-based agent (model-based, with a learning element)

- **Not simple reflex:** cameras give only the current picture; the best green time depends on queue history and flow trends, so the agent needs internal state.
- **Not just goal-based:** the controller has several conflicting objectives (low average waiting time, very high priority to emergency vehicles, fairness to all four approaches). A yes/no goal cannot say which signal plan is better; a **utility function** can trade these off numerically and choose the plan with the highest expected utility.
- **Model-based:** vehicle counts are only partially observable, so the agent keeps a world model (how traffic evolves and what each signal change will do to the queues).
- **Learning:** traffic patterns change through the day (peak hours, events), so a learning element tunes the model and utility weights to keep reducing the average waiting time.

ii. Agent architecture diagram



iii. Pseudocode for the decision cycle

```
function TRAFFIC-SIGNAL-AGENT(percept) returns action
  persistent: state, model, utility_weights, last_action

  state <- UPDATE-STATE(state, last_action, percept, model)
    // queue length per lane, arrival rate, emergency
lane

  if EMERGENCY-DETECTED(state) then
    action <- PREEMPT(state.emergency_lane)    // give
green, clear junction
  else
    best_utility <- -infinity
    for each phase in PHASES do                // N-S, E-W,
left-turns ...
      for each d in {10, 20, 30, 40, 50, 60} do // green
duration (sec)
        predicted <- RESULT(model, state, phase, d)
        U <- -(w1*AVG-WAIT(predicted)
              + w2*MAX-QUEUE(predicted)
              + w3*EMERGENCY-DELAY(predicted))
        if U > best_utility then
          best_utility <- U; action <- (phase, d)

  EXECUTE(action)                                // set signal heads
  reward <- MEASURE-WAITING-TIME()               // critic feedback
  LEARN(model, utility_weights, state, action, reward)
  last_action <- action
  return action
```

12. Mars Rover – Analysing and Exploring Samples on Mars

i. Percepts

- Stereo / hazard-camera images of terrain, rocks and soil; panoramic and microscopic images.
- Spectrometer, X-ray and other instrument readings giving mineral and chemical composition of samples.
- Environmental readings: temperature, atmospheric pressure, wind, dust level, radiation.
- Internal and position sensors: wheel odometry, IMU (tilt, slope, orientation), sun sensor, battery / solar-power level, wheel motor current (slip), arm position.
- Commands and data windows from Earth or an orbiter.

ii. Operating environment

Property	Value	Reason
Observable	Partially	Camera range is limited; terrain underneath, hidden rocks and sample contents are unknown
Deterministic?	Stochastic	Wheel slip, dust storms, unpredictable terrain and instrument noise
Episodic?	Sequential	Each move or drilling decision affects energy, position and later options
Static?	Dynamic	Weather, dust, lighting and temperature change day to night while the rover acts
Discrete?	Continuous	Position, speed, joint angles and sensor values are continuous
Agents	Single-agent	Mainly the rover alone (orbiter and ground control cooperate, no competitors)
Known?	Unknown	Terrain is unexplored; communication delay (several to ~20 minutes) forces autonomy

iii. Actions

- Drive forward / backward, steer, turn, stop, avoid obstacles, climb or reverse out of a slope.
- Deploy the robotic arm, scoop soil, drill or grind rock, place the sample in an instrument, run the analysis, store or discard the sample.
- Take and compress images, point the camera or antenna, adjust solar panels, enter sleep / recharge mode.
- Transmit results to Earth or an orbiter and receive commands.

iv. Performance measure

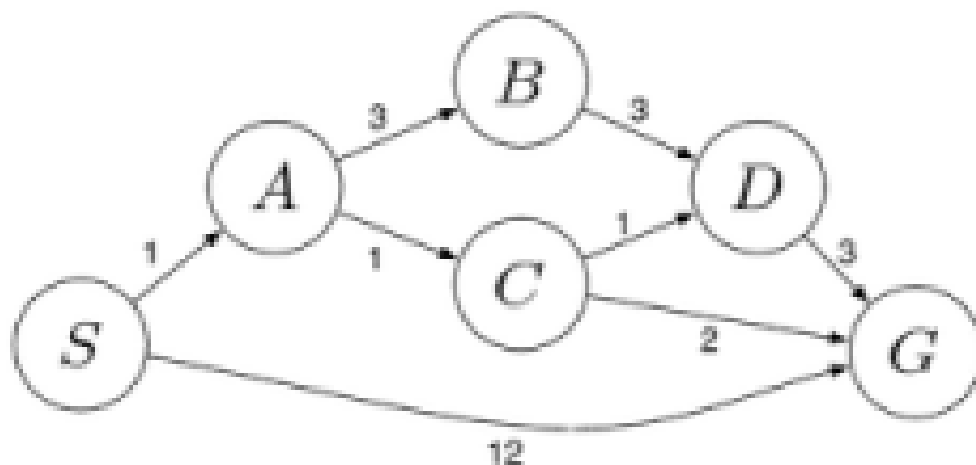
- Number and scientific value of distinct samples analysed (variety of rock and soil types, search for water and signs of life).
- Quality and accuracy of data returned to Earth, and area or distance explored.
- Safety: no collisions, tip-overs, getting stuck or instrument damage; mission lifetime.
- Efficiency in energy and time used per sample.

v. Most suitable agent architecture: utility-based learning agent (with a reactive safety layer)

- **Partially observable:** it needs a model-based agent that maintains a map and internal state.
- **Conflicting objectives:** science value vs. energy use vs. risk of damage, which a utility function can balance (a plain goal cannot).
- **Communication delay:** the rover cannot wait for Earth, so it must plan and decide autonomously.

- **Unknown, changing terrain:** a learning element improves its terrain and sample-value models over time.
- A simple reflex layer sits underneath for instant hazard avoidance (stop at a cliff or high tilt).

15. Uniform Cost Search for Least-Cost Delivery Route ($S \rightarrow G$)



Edges read from the graph (directed, as drawn): $S \rightarrow A = 1$, $S \rightarrow G = 12$, $A \rightarrow B = 3$, $A \rightarrow C = 1$, $B \rightarrow D = 3$, $C \rightarrow D = 1$, $C \rightarrow G = 2$, $D \rightarrow G = 3$.

UCS keeps a frontier (priority queue) ordered by path cost $g(n)$, always expands the cheapest node, and applies the goal test when a node is expanded (popped), not when generated. The entry shows node:g(parent).

Step	Node expanded	Explored set	Frontier after expansion (node : g)
1	S (0)	{S}	A:1(S), G:12(S)
2	A (1)	{S, A}	C:2(A), B:4(A), G:12(S)
3	C (2)	{S, A, C}	D:3(C), B:4(A), G:4(C) (replaces G:12, cheaper)
4	D (3)	{S, A, C, D}	B:4(A), G:4(C) [G via D = 6 is worse, ignored]
5	B (4)	{S, A, C, D, B}	G:4(C) [D via B = 7 is worse, ignored]
6	G (4)	—	Goal test succeeds – stop

(If G:4 is popped before B:4 because of the tie, the answer is the same.)

Result

- **Least-cost path:** $S \rightarrow A \rightarrow C \rightarrow G$
- **Total cost** = $1 + 1 + 2 = 4$.

Why this is optimal (comparison with other routes)

Route	Cost
$S \rightarrow G$	12
$S \rightarrow A \rightarrow B \rightarrow D \rightarrow G$	$1 + 3 + 3 + 3 = 10$
$S \rightarrow A \rightarrow C \rightarrow D \rightarrow G$	$1 + 1 + 1 + 3 = 6$
$S \rightarrow A \rightarrow C \rightarrow G$	$1 + 1 + 2 = 4$ (minimum)

UCS is complete and optimal when all step costs are positive, because nodes are expanded in non-decreasing order of path cost; the first time G is removed from the frontier its cost (4) is guaranteed minimal. The company should route packages from warehouse S via A and C to G at a cost of 4 units (fuel + time).